

Lập trình web 1- PHP&MySQL – Lab 11

Nội dung thực hành:

- Namespace & Autoload: Tổ chức class và tự động nạp file.
- Entry Point: Xử lý request và phân biệt Admin/Client.
- Authentication & Middleware: Kiểm tra đăng nhập, Guest và CSRF.
- AuthController: Xử lý Login/Logout.
- .htaccess: Rewrite URL, ẩn index.

Câu A. SỬ DỤNG LẠI PROJECT MiniShop_HoTen	2
Câu B. NHẬN XÉT LẠI SOURCE	2
Câu C. TỔ CHỨC LẠI CẤU TRÚC	2
Câu D. TẠO AUTOLOAD	4
Câu E. Thêm Namespace	6
Câu F. Kiểm tra Authentication / Middleware	8
Câu G. TẠO .HTACCESS	11
Câu H. KIỂM TRA TOÀN BỘ CRUD.....	12
Câu I. ĐĂNG SOURCE CODE THAY ĐỔI LÊN GITHUB.....	12

Lập trình web 1- PHP&MySQL – Lab 11

Câu A. SỬ DỤNG LẠI PROJECT MiniShop_HoTen

Nếu làm trên máy khác, clone phiên bản mới nhất của Project từ GitHub về máy tính bằng lệnh:

- git clone https://github.com/<username>/HoTenSVKhongDau_LTW1.git

Câu B. NHẬN XÉT LẠI SOURCE

1. Luồng xử lý (index.php)

a. Nhận Request

<http://localhost/miniproject/views/admin/products/index.php?limit=5&sort=>

b. **Xử lý Request** : Đọc limit, sort, page, keyword từ \$_GET.

c. **Gọi DAO**: Gọi ProductDAO/CategoryDao/... để lấy dữ liệu.

d. **Hiển thị dữ liệu** : Hiển thị danh sách và phân trang.

⇒ Hiện tại index.php đang thực hiện cả xử lý Request và hiển thị View

2. Các khó khăn:

a. Code xử lý và giao diện trộn lẫn

b. File index.php **dài và khó quản lý**

c. Khó **tái sử dụng** phần xử lý Request cho các chức năng khác

d. Khi thay đổi cách xử lý, dễ ảnh hưởng đến phần giao diện.

3. Giải pháp:

a. Tách phần xử lý Request ra Controller

b. View chỉ tập trung hiển thị dữ liệu.

Request ⇒ **Controller** ⇒ DAO ⇒ Database ⇒ **Controller** ⇒ View

Câu C. TỔ CHỨC LẠI CẤU TRÚC

1. Tạo thư mục:

<pre>controllers/ └── admin/ └── ProductController.php models/ dao/ views/ config/</pre>	models/: Các class Model dao/: Truy vấn Database controllers/: Xử lý Request và điều hướng views/: Giao diện config/: Cấu hình hệ thống
--	--

2. Sửa và kiểm tra code

Tạo class ProductController	controllers/admin/ProductController.php
Tạo action index()	Class ProductController{ //... public function index() { // Gán dữ liệu cho tiêu đề trang \$title = "Quản lý sản phẩm"; // Đọc request từ URL \$keyword = trim(\$_GET["keyword"] ?? ""); \$limit = (int)(\$_GET["limit"] ?? 10); \$page = (int)(\$_GET["page"] ?? 1); // Xử lý offset \$offset = (\$page - 1) * \$limit; // Gọi Dao \$productDAO = new ProductDAO(); \$totalRecords = \$productDAO->count("products", "productname", \$keyword); \$totalPages = ceil(\$totalRecords / \$limit); \$products = \$productDAO->getPage(\$limit, \$offset, \$keyword); // Gọi View require __DIR__ . "/../views/admin/products/index.php"; }
Di chuyển code trong views/admin/index.php: + Nhận request + Gọi Dao	

3. Tạo index.php dùng chung để điều hướng (Route)

- File index.php tiếp nhận Request và điều hướng đến Controller tương ứng

MINISHOP_HOTEN/ └─index.php └─.....	Nhận Request. Đọc thông tin chức năng cần truy cập. Xác định Controller. Xác định Action. Gọi Controller tương ứng.
---	---

index.php?controller=product&action=index ProductController => index()	<?php require_once __DIR__ . "/controllers/admin/ProductController.php"; require_once __DIR__ . "/controllers/admin/CategoryController.php"; //... // Nhận Request \$controller = \$_GET["controller"] ?? "product"; \$action = \$_GET["action"] ?? "index"; // Xác định tên Controller \$controllerClass = ucfirst(\$controller) . "Controller"; // Kiểm tra Controller if (!class_exists(\$controllerClass)) { die("Controller không tồn tại"); } // Tạo Controller \$controllerObject = new \$controllerClass(); // Kiểm tra Action if (!method_exists(\$controllerObject, \$action)) { die("Action không tồn tại"); } // Gọi Action \$controllerObject->\$action();
index.php?controller=category&action=index CategoryController => index()	

4. Chạy và kiểm tra index.php

- Nên tắt AuthMiddleware trong master.php trước khi test (*hãy mở lại khi kiểm tra xong*)
- Truy cập:
[http://localhost\[:port\]/MiniShop_Hoten/index.php?controller=product&action=index](http://localhost[:port]/MiniShop_Hoten/index.php?controller=product&action=index)
- Chụp màn hình kết quả kiểm tra (hình chụp có hiển thị URL trên trình duyệt) và lưu vào Lab11_Capture.docx

5. Commit kết quả đã thực hiện

Câu D. TẠO AUTOLOAD

1. Ý nghĩa:

- Tự động nạp các class như Controller, DAO, Model, Middleware khi cần sử dụng.
- Không phải `require_once` từng file.

2. Tạo autoload.php

MINISHOP_HOTEN/ └──index.php └──autoload.php └──.....	Nhận Request. Đọc thông tin chức năng cần truy cập. Xác định Controller. Xác định Action. Gọi Controller tương ứng.
--	---

autoload.php	
<pre><?php spl_autoload_register(function (\$className) { \$folders = ['controllers/Admin', 'controllers/Client', 'controllers', 'dao', 'models', 'middleware', 'config']; foreach (\$folders as \$folder) { \$file = __DIR__ . '/' . \$folder . '/' . \$className . '.php'; if (file_exists(\$file)) { require_once \$file; return; } } });</pre>	+ spl_autoload_register() → đăng ký chức năng tự động load class. + \$className → tên class cần tìm. + \$folders → danh sách các thư mục cần tìm. + __DIR__ → thư mục hiện tại của autoload.php. + file_exists() → kiểm tra file có tồn tại. + require_once → nạp file nếu tìm thấy. + return → dừng tìm kiếm khi đã tìm thấy file.

- Nhúng autoload vào index.php (Entry point):
 - `require_once __DIR__ . '/autoload.php';`
- Xóa** các `require` / `require_once` không cần thiết trong *Controller, DAO, Model, Middleware* vì Autoload đã tự động nạp class.
- Chạy và kiểm tra:
 - Truy cập [http://localhost\[:port\]/MiniShop_Hoten/index.php?controller=product&action=index](http://localhost[:port]/MiniShop_Hoten/index.php?controller=product&action=index)
 - Trang Product hoạt động bình thường => **Autoload** đã tự động nạp các class cần thiết
 - Nếu báo **“Controller không tồn tại”** → kiểm tra lại đường dẫn ProductController.php trong controllers/Admin/ và cấu hình Autoload.

Câu E. Thêm Namespace

controllers/ ├── admin/ │ └── ProductController.php └── client/ └── ProductController.php	Khó phân biệt => cần Namespace để phân biệt
---	---

- **namespace** dùng để phân nhóm, tổ chức class và tránh trùng tên giữa các class.
- **use** dùng để gọi/import class từ namespace khác.

Admin/ ProductController.php	ProductDao.php; BaseDao.php
namespace Controllers\Admin; use DAO\ProductDAO; class ProductController { }	namespace DAO; class ProductDAO { }

- Thêm xử lý lại index.php(entry point) cho phù hợp

```
<?php
require_once __DIR__ . '/autoload.php';
// Nhận Request
$area = $_GET["area"] ?? "client";
$controller = $_GET["controller"] ?? "product";
$action = $_GET["action"] ?? "index";
// Xác định tên Controller
$controllerClass = ucfirst($controller) . "Controller";
// Xác định tên Controller
if ($area === "admin") {
    $controllerClass = "Controllers\\Admin\\" . ucfirst($controller) . "Controller";
} else {
    $controllerClass = "Controllers\\Client\\" . ucfirst($controller) . "Controller";
}
// Kiểm tra Controller
// Tạo Controller
// Kiểm tra Action
// Gọi Action
```

- Update xử lý lại autoload.php cho phù hợp (có sử dụng namespace)

<pre><?php spl_autoload_register(function (\$className) { \$prefixes = ['Controllers\\' => __DIR__ . '/controllers/', 'DAO\\' => __DIR__ . '/dao/', 'Models\\' => __DIR__ . '/models/', 'Middleware\\' => __DIR__ . '/middleware/', 'Config\\' => __DIR__ . '/config/',]; foreach (\$prefixes as \$prefix => \$baseDir) { // Kiểm tra class có thuộc namespace này không if (str_starts_with(\$className, \$prefix)) { // Bỏ phần namespace gốc \$relativeClass = substr(\$className, strlen(\$prefix)); // Đổi dấu \ thành / \$file = \$baseDir . str_replace('\\', '/', \$relativeClass) . '.php'; // Nếu file tồn tại thì nạp file if (file_exists(\$file)) { require_once \$file; } return; } } });</pre>	namespace Controllers\Admin; Controllers\Admin\ProductController Autoload sẽ chuyển thành: controllers/admin/ProductController.php DAO\ProductDAO ↓ dao/ProductDAO.php Models\Product ↓ models/Product.php
--	---

- Kiểm tra Namespace và thư viện PHP:
 - o Kiểm tra lại toàn bộ các file bên dưới và khai báo **Namespace đầy đủ**,
 - config/Database.php
 - dao/BaseDAO.php
 - dao/ProductDAO.php
 - models/Product.php
 - o Kiểm tra các use tương ứng giữa các Namespace.
- Hãy kiểm tra lại toàn các file BaseDao.php, Database, productdao, ,model/product đã thêm đầy đủ namespace chưa
- Đối với các thư viện/class có sẵn của PHP như mysqli, PDO, Exception, DateTime..., khi sử dụng trong Namespace cần thêm \ phía trước
 - o new \mysqli(...);
 - o catch (\Exception \$e)
 - o Chạy và kiểm tra: (thêm **area**)

- Truy cập
[http://localhost\[:port\]/MiniShop_Hoten/index.php?area=admin&controller=product&action=index](http://localhost[:port]/MiniShop_Hoten/index.php?area=admin&controller=product&action=index)
- Trang Product hoạt động bình thường => **Autoload** đã tự động nạp các class cần thiết
Nếu báo “**Class ... not found**” → kiểm tra Namespace, use và Autoload của class đó.

- Commit kết quả vừa thực hiện

Câu F. Kiểm tra Authentication / Middleware

1. Áp dụng namespace vào AuthMiddleware.php
2. Update lại đường dẫn trong file AuthMiddleware.php

```
class AuthMiddleware
{
    public static function handle()
    {
        if (session_status() === PHP_SESSION_NONE) {
            session_start();
        }
        if (!isset($_SESSION["user"])) {
            header("Location: index.php?area=admin&controller=auth&action=login");
            exit;
        }
    }
}
```

3. Chuyển xử lý AuthMiddleware từ master.php vào index.php(entry point)

master.php	index.php(entry point)
<pre>session_start(); AuthMiddleware::handle();</pre>	<pre><?php require_once __DIR__ . '/autoload.php'; session_start(); // Nhận Request \$area = \$_GET["area"] ?? "admin"; \$controller = \$_GET["controller"] ?? "product"; \$action = \$_GET["action"] ?? "index"; // *** Kiểm tra Authentication cho Admin if (\$area === "admin" && \$controller !== "auth") { \Middleware\AuthMiddleware::handle(); } // *** Kiểm tra Guest if (\$area === "admin" && \$controller === "auth" && \$action === "login") { \Middleware\GuestMiddleware::handle(); }</pre>

	<pre> } // *** Tạo CSRF Token if (\$_SERVER["REQUEST_METHOD"] === "POST") { \Middleware\CsrfMiddleware::generateToken(); } // </pre>
--	---

4. Tạo AuthController (controller/admin/AuthController.php)

- Di chuyển code đọc request , gọi request trong login.php vào function login() AuthController.php
- Di chuyển code đăng xuất trong logout.php vào function logout()

<pre> <?php namespace Controllers\Admin; use DAO\UserDAO; use Middleware\CsrfMiddleware; class AuthController { public function login() { \$errors = []; // Hiển thị form if (\$_SERVER["REQUEST_METHOD"] === "GET") { require_once __DIR__ . '/../views/admin/login.php'; return; } // Kiểm tra CSRF CsrfMiddleware::verify(); // Nhận dữ liệu \$username = trim(\$_POST["username"] ?? ""); \$password = \$_POST["password"] ?? ""; // Validate if (\$username === "") { \$errors['username'] = "Vui lòng nhập tên đăng nhập."; } if (\$password === "") { \$errors['password'] = "Vui lòng nhập mật khẩu."; } // Nếu có lỗi if (!empty(\$error)) { require_once __DIR__ . '/../views/admin/login.php'; return; } // Tìm User \$userDAO = new UserDAO(); </pre>	+ login() → hiển thị trang đăng nhập. + logout() → đăng xuất và hủy Session ** Do đã di chuyển xử lý từ file logout.php vào controller nên xóa file logout.php (không sử dụng)
--	--

```
$user = $userDAO->findByUsername($username);

// Kiểm tra tài khoản và mật khẩu
if(!$user){
    $errors['username'] = "Tên đăng nhập không đúng.";
    require_once __DIR__ . '/../views/admin/login.php';
    return;
}
else if (!password_verify($password, $user->password)) {
    $errors['password'] = "Mật khẩu không đúng.";
    require_once __DIR__ . '/../views/admin/login.php';
    return;
}
// Đăng nhập thành công
$_SESSION["user"] = $user;

header("Location:
index.php?area=admin&controller=product&action=index");
exit;
}

// đăng xuất
public function logout()
{
    session_unset();
    session_destroy();

    header("Location:
index.php?area=admin&controller=auth&action=login");
    exit;
}
}
```

5. Chạy và kiểm tra

- Truy cập: /MiniShop_Hoten/index.php?area=admin&controller=auth&action=login
- Kiểm tra tài khoản sai => hiển thị lỗi
- Kiểm tra đăng nhập thành công => vào trang dashboard
- Kiểm tra AuthMiddleware:
 - Nhấn đăng xuất
 - truy cập trực tiếp:
/MiniShop_Hoten/index.php?area=admin&controller=product&action=index
 - Kết quả: Chưa đăng nhập → chuyển về trang Login.
- Kiểm tra Logout:
 - Truy cập: `index.php?area=admin&controller=auth&action=logout`
 - Kết quả: Hủy session → chuyển về trang Login.

Câu G. TẠO .HTACCESS

1. file .htaccess:

- .htaccess được dùng để điều khiển cách Apache xử lý request
- Rewrite URL → tạo URL ngắn, đẹp và dễ nhớ.
- Đưa mọi request về index.php (Entry Point)
- Ẩn index.php khỏi URL.
- Giúp URL dễ đọc, thân thiện hơn.

/admin/product/edit/5 => .htaccess => index	
index.php?area=admin&controller=product&action=index	/admin/product
index.php?area=admin&controller=product&action=edit&id=5	/admin/product/edit/5
index.php?area=admin&controller=product&action=create	/admin/product/create
index.php?area=admin&controller=auth&action=login	/admin/login
.....	
index?area=admin&controller=product&action=index	/admin/product
index?area=client&controller=product&action=index	/product

```
RewriteEngine On
RewriteBase /Minishop_Hoten/
# Không rewrite file và thư mục có thật
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
# Đưa mọi request về Entry Point index.php
RewriteRule ^ index.php [QSA,L]
```

2. Chạy và kiểm tra:

- Truy cập http://localhost:93/Minishop_Hoten/
- Truy cập http://localhost:93/Minishop_Hoten/admin/login
- Truy cập http://localhost:93/Minishop_Hoten/admin/product

3. Commit kết quả thực hiện

Câu H. KIỂM TRA TOÀN BỘ CRUD

1. Kiểm tra toàn bộ chức năng CRUD của các đối tượng trong hệ thống:
 - a. Sản phẩm
 - b. Danh mục
 - c. Thương hiệu
 - d.
2. Nội dung kiểm tra:
 - a. Create: Thêm mới dữ liệu.
 - b. Read: Hiển thị danh sách và xem chi tiết.
 - c. Update: Chỉnh sửa và cập nhật dữ liệu.
 - d. Delete: Xóa dữ liệu.
 - e. Kiểm tra Validation khi nhập dữ liệu.
 - f. Kiểm tra CSRF Token đối với các thao tác thay đổi dữ liệu.
 - g. Kiểm tra Authentication / Middleware đối với khu vực Admin.
 - h. Kiểm tra dữ liệu sau thao tác có được cập nhật chính xác trong Database hay không.
3. Chạy và kiểm tra:
 - a. Các chức năng CRUD hoạt động bình thường

Câu I. ĐẨY SOURCE CODE THAY ĐỔI LÊN GITHUB

- Mở terminal tại thư mục chứa project
- Thêm source vào Git, dùng lệnh : git add .
- Commit source: git commit -m "MiniProject - X"
- Đẩy source lên GitHub: git push origin main
- Kiểm tra lại trên GitHub: source code đã được cập nhập